CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

AI Usage Log

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

Format per Annexure A of the course brief.

Declaration

This project was built with substantial AI assistance. The tool used throughout was **Claude (Anthropic), via the Claude Code CLI**, which can read and write files in the repository and run commands, so its output was not only text but working code, spreadsheets and documents.

The log below records the significant interactions. It was maintained during the work, not reconstructed afterwards, and it includes the occasions on which the AI was **wrong**, the occasions on which **we were wrong and the AI was right**, and the occasions on which its output was **rejected on principle** even though it would have worked.

Entries marked **Modify** or **Reject** are the ones worth reading. A log consisting entirely of "Accept" would indicate that nobody was checking.

The **Member** column records which of us ran the interaction and carried out the verification in the final column, assigned by the role allocation in the project charter. Where two of us worked on an entry together the owner of the deliverable is named.

Verification standard applied throughout: no AI-produced number entered a deliverable until it had been reproduced either by hand or by an independent implementation. The 50-comparison benchmark in `docs/04-validation-report.md` is the formal expression of this.

---

1. Log

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
1	Sajjad	2026-10-29	Claude	Read the course brief; assess which of the 14 topics best fits a team that wants to demonstrate method depth rather than breadth	Recommended Topic 7 (EVM); noted that the brief's own validation requirement ("manually compute all EVM indices for two reporting periods") is unusually well suited to a rigorous two-implementation comparison	**Accept**	Reasoning checked against the brief's assessment weights: validation is 20%, the largest single component alongside tool functionality

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
2	Sajjad	2026-10-29	Claude	Review `Tahir-yamin/excel-scurve-builder` as prior art and identify what to adopt and what to improve on	Identified three adoptable ideas (live formulas, validation-by-execution, Excel chart defaults) and one clear gap (it rolls up to a single curve and lists the absence of a WBS breakdown as a known limitation)	**Accept**	Read the repository's README and gotchas document directly rather than relying on the summary. Credited in `docs/02-method-note.md` §3.1
3	Dependar	2026-10-29	Claude	Design a case study project that exercises every branch of the EVM method	G+4 RCC academic block, ₹14 crore, 19 work packages, 12 periods, built around a hard-rock excavation overrun cascading into the frame	**Modify**	First draft had the variance spread evenly across packages, which makes the WBS breakdown pointless. Reworked so 74% of the adverse variance sits in three completed substructure packages — the distinction between recoverable and final variance is the thing worth demonstrating
4	Dependar	2026-10-29	Claude	Choose a phasing curve for work packages within their baseline window	Proposed smoothstep `3t² − 2t³`, with zero slope at both ends	**Accept**	Verified the aggregate of 19 staggered packages produces a credible project S-curve, and that cumulative PV lands on exactly ₹14.00 crore. The S-curve is emergent from the phasing, not imposed
5	Neel	2026-10-29	Claude	Implement the core EVM measures in Python	`evm/core.py` with PV/EV/AC matrices and a `PeriodMetrics` dataclass	**Accept**	Smoke-tested against a hand calculation before any further work (entry 9)
6	Neel	2026-10-29	Claude	Review the progress carry-forward routine	Flagged its own earlier code: nested loop with a conditional indexing a pivot table by a column that might not exist	**Modify**	Replaced with forward-fill then zero-fill. Had not yet produced a wrong answer because the case study has no reporting gaps — but real data does. Recorded as discrepancy 5.4 in the validation report
7	Sajjad	2026-10-29	Claude	Explain why classical SPI is unreliable and whether it should be supplemented	Explained the convergence defect (EV → BAC and PV → BAC at completion, so SPI → 1.00 however late) and proposed Earned Schedule	**Accept**	Verified independently: on the case study SPI reads 0.863 and SPI(t) reads 0.893, and SV(t) of −0.64 months converts to a statement a site manager can act on
8	Sajjad	2026-10-29	Claude	Implement the four standard EAC methods	`evm/forecasting.py` with EAC1–4, ETC, VAC, TCPI	**Modify**	Initial version returned a single "best" EAC. Rejected that design: a single EAC implies a precision the method does not have. Changed to return all four with their assumptions stated, and the dashboard presents the range
9	Akshat	2026-10-29	— (manual)	Hand-calculate PV, EV, AC and all indices at Period 6 from the raw inputs	PV ₹4,02,48,444; EV ₹3,47,30,000; AC ₹3,98,50,000; CPI 0.871518; SPI 0.862890	**Accept**	**Done by hand before running the engine.** Engine then matched to six decimal places
10	Akshat	2026-10-29	Claude	Write a test suite whose expected values are the hand figures from entry 9	34 tests in `tests/test_core.py` with the derivations in the docstrings	**Accept**	All passed. Hand derivations written into the tests so the arithmetic can be followed in the viva without re-running anything

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
11	Akshat	2026-10-29	Claude	Add algebraic identity tests	Proposed TCPI-against-EAC1 = CPI, and VAC-on-EAC2 = CV	**Accept**	Genuinely valuable suggestion we had not thought of. These hold only if the whole forecast chain is intact, so they catch an error that per-value comparison can miss
12	Akshat	2026-10-29	Claude	**Investigate: hand calculation of EAC3 disagrees with engine by ₹156**	Recomputed with 40-digit decimal arithmetic; showed the engine matched full precision and the hand figure had rounded CPI × SPI to six decimal places mid-chain	**Accept**	**The AI was right and we were wrong.** Verified independently with Python's `decimal` module at 40 digits. Corrected the hand figure, adopted "rounding is for presentation only" as a rule, and pinned it with a regression test. Recorded as discrepancy 5.1
13	Akshat	2026-10-29	Claude	Two input-validation tests are failing — diagnose	Determined that in both cases an earlier validation rule fired before the rule under test, so the engine was correct and the tests were mis-constructed	**Accept**	**Resisted the tempting fix.** Changing the engine to make the tests pass would have removed a correct validation rule. Rewrote both tests to exercise each rule in isolation; no engine code changed. Recorded as discrepancy 5.2
14	Sajjad	2026-10-29	Claude	Should the monthly commentary be generated by a language model?	Argued **against** its own use: a cost report must be reproducible month to month, must not contain numbers that were never computed, and must have arguable thresholds	**Accept**	Adopted. Commentary is template-driven and deterministic. A test asserts every figure in the output text is one the engine computed. This is the project's central argument about AI in practice
15	Pearsh	2026-10-29	Claude	Draft the commentary templates	Four paragraphs: cost, schedule, diagnosis by package, forecast	**Modify**	Changed the index band wording from "behind plan" to "below plan". A cost index is never "behind" — it was schedule idiom applied to a cost measure, which would have read as sloppy to a practitioner
16	Akshat	2026-10-29	Claude	Design the manual benchmark workbook so that it is genuinely independent	Proposed typed Excel formulas acting on raw inputs, with no value carried across from Python	**Accept**	Checked the independence claim carefully: the two sides share *inputs* but not *calculations*. Stated explicitly in the workbook's Read Me and in validation report §6
17	Akshat	2026-10-29	Claude	The benchmark workbook's cells are empty when read back — diagnose	openpyxl writes formulas but no cached results; a reader trusting the cache sees nothing. Fix: set `wb.calculation.fullCalcOnLoad = True`	**Accept**	Verified by round-tripping through LibreOffice and reading 25 computed values per period
18	Akshat	2026-10-29	Claude	How should the Excel benchmark be compared against the Python engine?	Recommended **executing** the workbook via LibreOffice headless rather than re-implementing the formulas in Python, citing the prior-art argument that a Python re-implementation validates only our own understanding	**Accept**	This reshaped the validation strategy and is the single most consequential suggestion in the project. Had we re-implemented, a misread formula would have been misread identically on both sides and the validation would have passed
19	Akshat	2026-10-29	Claude	Run the full benchmark comparison	50 of 50 comparisons agree, to ₹0.01 on currency and 1e−9 on indices, across both periods	**Accept**	Output inspected measure by measure rather than taken on the summary line. The identity checks (TCPI/EAC1 = CPI) hold on the Excel figures too

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
20	Neel	2026-10-29	Claude	Apply the chart-default fixes documented in the prior-art repository	Four overrides: `x_axis.auto = False`, `invertIfNegative = False`, `tickLblPos = "low"`, and `NA()` instead of `""` beyond the data date	**Accept**	Each is invisible to XML inspection — an element you never wrote has no XML to read back — so added a test asserting each override is set, and opened the workbook to confirm the charts are legible
21	Neel	2026-10-29	Claude	Build the live-formula Excel export	Workbook with Dashboard/Inputs/Baseline/EVM/Variance sheets, EV computed by `SUMPRODUCT` over budgets and percentages	**Accept**	Verified by evaluating the export through LibreOffice: every figure matches the engine to the paisa. Kept strictly separate from the benchmark — a workbook the tool generated cannot validate the tool
22	Neel	2026-10-29	Claude	Lay out the Streamlit dashboard	Proposed: verdict and plain-language summary first, then indices, then diagnosis, then forecast	**Accept**	Ordering defended on the grounds that an EAC without knowing what drives it gives a manager nothing to act on. Checked by opening the running app and reading it as a user would
23	Neel	2026-10-29	Claude	**Bug found by looking at the running app: the executive summary is invisible**	Custom CSS set light panel backgrounds without setting a text colour; under a dark browser preference Streamlit supplied white text, rendering white-on-white	**Accept**	**No unit test would have caught this.** Every number was correct and the tool was unusable. Fixed with explicit colours and a pinned light theme. Recorded as discrepancy 5.3
24	Neel	2026-10-29	Claude	Set charting conventions for the dashboard	Curves stop at the data date; forecast dashed and branching; index axes pinned not auto-scaled	**Accept**	Each convention has a stated reason in `app/charts.py`. The pinned axis matters most: an auto-scaled index axis makes the same CPI look catastrophic or negligible depending on the rest of the series
25	Neel	2026-10-29	Claude	Review the S-curve forecast tail	Axis label read `+7.4p`, which is cryptic	**Modify**	Relabelled to `forecast finish ~P13.4` and moved the EAC value into the legend entry
26	Dependar	2026-10-29	Claude	Review the case study for realism before submission	Flagged that the baseline ran Apr-2025 to Mar-2026, putting the "current" data date a year in the past at submission	**Accept**	Shifted to Apr-2026 – Mar-2027 (data date Sep-2026). Numerically identical; benchmark rebuilt and revalidated at 50/50, screenshots retaken
27	Pearsh	2026-10-29	Claude	Draft the method note, user manual and validation report	Full drafts	**Modify**	Substantially restructured. The AI's first draft of the validation report led with the 50/50 result; moved "what this validation does **not** establish" from an appendix to a prominent section, because it is the more important finding
28	Pearsh	2026-10-29	Claude	Write the limitations sections	Comprehensive list across manual, charter and workbook	**Accept**	Verified each limitation is genuinely true of this implementation rather than generic hedging. The percent-complete limitation is repeated in four places deliberately
29	Pearsh	2026-10-30	Claude	Draft the learnings and reflection report against the six sections the brief specifies	Full draft, including the AI capability assessment and the ethics and limitations sections	**Modify**	Rewrote §2 substantially. The draft's assessment of where AI was weakest was generic ('struggles with ambiguity'); replaced with the two specific failures we actually observed — it could not detect the invisible-text bug because that is not a property of code, and it proposed a validation strategy that would have validated our own misunderstanding until the prior-art argument was put to it

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
30	Pearsh	2026-10-30	Claude	Draft the presentation slide content, demo script and viva preparation	12 slides with timings, a demonstration checklist and ~30 anticipated questions with answers	**Modify**	Checked every figure quoted on a slide against the engine output rather than against the draft. Added the algebraic proof that TCPI against EAC1 reduces to CPI, because an examiner asking why two of our reported measures are identical is a question we should not have to derive on the spot
31	Pearsh	2026-10-30	Claude	Draft scaffolds for the five individual reflections in §5 of the reflection report, from each member's role and the discrepancies attributable to it	Five first-person half-page entries against a common four-question structure	**Modify**	**The most important verification in this log.** An AI-drafted account of a person's own learning is not that person's reflection, and submitting one as such would be exactly the misrepresentation the brief's academic-integrity clause describes. The drafts were used as a structure and a prompt only; each member rewrote their own entry in their own words and confirmed its content before submission. The factual anchors in each — the EAC3 rounding error, the evenly-spread case study, the dark-theme bug, the two mis-constructed tests — are drawn from the discrepancy log and are matters of record
32	Neel	2026-11-05	Claude	Add a data-upload route to the dashboard so a reviewer can run the tool on their own project, plus a template download	A sidebar data-source selector, a four-file uploader writing to a temporary folder, and a zipped copy of the case study as the editable template	**Accept**	Tested by loading the four case-study files through the upload path and confirming the engine returned CPI 0.8715 at Period 6, identical to the folder path. The template ships the *worked* case study rather than blank files deliberately: a baseline that must reconcile to each package budget, and packages that must sum to BAC, is far easier to preserve by editing than to achieve from an empty form
33	Akshat	2026-11-05	Claude	Generate a second case study in the same four-file format, to test the upload route	A 15 MLD water treatment plant, 13 packages, BAC Rs 8.50 crore, deliberately under budget and behind programme	**Modify**	Accepted the data after checking it loads and that the engine returns CPI 1.041 and SPI 0.827 as designed. The run immediately exposed a defect in our own commentary: a project forecast to underrun was told it needed a **-5% improvement** in cost efficiency, because the closing sentence was generated unconditionally from TCPI over CPI. Fixed, logged as discrepancy 5.5, and pinned by a new regression test. Worth recording that the case study was built to exercise untested branches and found a defect on its first run

2. Notable entries expanded

The table above satisfies the format. These five entries are where the critical judgement actually happened.

2.1 Entry 12 — the AI was right and our hand calculation was wrong

Our hand calculation of EAC method 3 gave ₹17,98,32,249. The engine gave ₹17,98,32,093, a difference of ₹156.

The instinct on finding a discrepancy between a hand check and a tool is that the tool is wrong — that is, after all, the point of a hand check. Instead of assuming, the quantity was recomputed in 40-digit decimal arithmetic. The engine matched full precision exactly. We had rounded the composite index CPI × SPI to six decimal places partway through, which is what one naturally does on paper and which looks harmless; because EAC3 divides a ₹10.5 crore remainder by that index, it is not.

Three things came out of this that would not have come out of simply accepting the engine's number:

1. A rule for the rest of the project: **rounding belongs in presentation only, never in the calculation chain.** The benchmark workbook now rounds through cell number formats alone.
2. A regression test that fails if anyone reintroduces mid-chain rounding.
3. A genuine piece of subject learning. ₹156 on an ₹18 crore forecast is immaterial, but the same habit applied to CPI early in a project — when EV is small and the divisor volatile — moves the forecast by lakhs.

The AI's contribution was the investigation method, not the answer. Asking "which of these two is right, and prove it independently" is the move that mattered, and it is available to anyone.

2.2 Entry 13 — resisting the easy fix

Two validation tests failed. The fastest route to a green test suite would have been to adjust the engine until they passed.

Diagnosis showed the engine was correct in both cases: an earlier validation rule was firing before the rule under test, because the test data was broken in two ways at once. "Fixing" the engine would have meant **deleting a correct validation rule to satisfy a badly written test** — and the rule in question rejects progress reported against a summary WBS branch, which is a real way for earned value to be double-counted.

This is the failure mode the brief warns about, in miniature. The AI produced both the mis-constructed tests and the correct diagnosis. What determined the outcome was asking *why* the test failed before changing anything.

2.3 Entry 14 — rejecting an AI feature on principle

The brief asks for "automatic plain-language monthly progress commentary", and the obvious implementation is to pass the indices to a language model. We asked whether we should, and the argument that came back was against it. We adopted that argument:

- **Reproducibility.** The same inputs must produce the same sentence every month. "The model phrased it differently this time" is not a defence in front of a client or a lender.
- **It cannot invent.** A language model given a table of indices will readily produce a fluent paragraph containing a number that is not in the table. In `evm/commentary.py` there is no path by which a figure reaches the text without having been computed, and `test_every_figure_quoted_is_a_computed_figure` asserts it.
- **Auditability.** Every sentence traces to a threshold written down and justified in one file, where a reader who disagrees can find it.

The tool is therefore *less* AI-powered than it could be, and the reduction is the point. The place AI earned in this project is upstream — drafting those templates, checking formula logic, writing code, catching our arithmetic slip — not generating the numbers a decision rests on.

2.4 Entry 23 — what no test catches

The dashboard's executive summary rendered white-on-white under a dark browser theme. Every figure was correct. All 128 tests at the time passed. The tool was unusable.

It was found by opening the running application and looking at it.

The lesson is narrow and worth stating because it cuts against the rest of this project's emphasis: validation establishes that the arithmetic is right. It says nothing about whether the thing works. "Validated" and "usable" are different claims, and only one of them can be automated.

2.5 Entry 31 — the limit of what AI could write for us

The five individual reflections in §5 of the reflection report are the one deliverable where AI assistance is constrained by what the deliverable *is*, rather than by whether the output would be any good.

Claude can write a fluent, specific, plausible account of what a person learned, because it has the discrepancy log and the role allocation and can infer what the Validation Engineer must have found difficult about a rounding error. The output reads well. It is also, as a reflection, worth nothing: a reflection is a first-person claim about a particular person's understanding, and nobody but that person can make it. Submitting an AI-written account of our learning as our

learning would be the precise misrepresentation the brief's academic-integrity clause describes, and it would collapse the first time an examiner asked one of us about our own paragraph.

We used the drafts as a structure and a prompt — the four questions, and a reminder of which recorded discrepancy each of us owned — and each member then wrote their own entry. The factual anchors are matters of record from the discrepancy log; the understanding described is each member's own.

This is the clearest case in the project of AI output being useful at one remove: valuable as scaffolding, inadmissible as product.

3. Where AI helped, and where it did not

Genuinely accelerated

- Boilerplate: data loading, pivoting, Excel layout, Streamlit scaffolding. Hours of typing, little judgement.
- Recall of standard formulas, correctly, including the less-used ones (TCPI against EAC, the weighted EAC4).
- Operational knowledge that would otherwise cost a day of debugging — `fullCalcOnLoad`, the Excel chart defaults, the LibreOffice headless invocation.
- Adversarial review of its own earlier output (entry 6).
- Arithmetic verification at a precision we would not have attempted by hand (entry 12).

Did not help, or needed correcting

- **Judgement about what matters.** Its first validation report led with "50 of 50 agree". The more important finding is that both implementations share inputs, so neither can detect a wrong percent-complete figure. We moved that to a prominent section; it did not propose doing so.
- **Domain idiom.** "Behind plan" for a cost index (entry 15) is the kind of error that signals to a practitioner that the author has not worked on a site.
- **Case study realism.** Its first draft spread variance evenly across packages, which would have made the central feature pointless.
- **Anything visual.** Entry 23.

The pattern. AI was strongest where the answer is determinate and checkable — a formula, a library call, a precision question. It was weakest where the question is "what matters here?", which is most of engineering judgement and all of professional responsibility.

---

4. Accountability

Every number in every deliverable has been verified against an independent manual benchmark, and the comparison is reproducible by anyone with the repository and LibreOffice.

Where AI output was accepted, it was accepted after checking. Where it was wrong, that is recorded here and in the validation report. Where we were wrong and it was right, that is recorded too.

The team is accountable for all of it. "The AI generated it" is not a defence, and the main reason to keep a log in this much detail is that writing down *why* each output was accepted forces the check to actually happen.